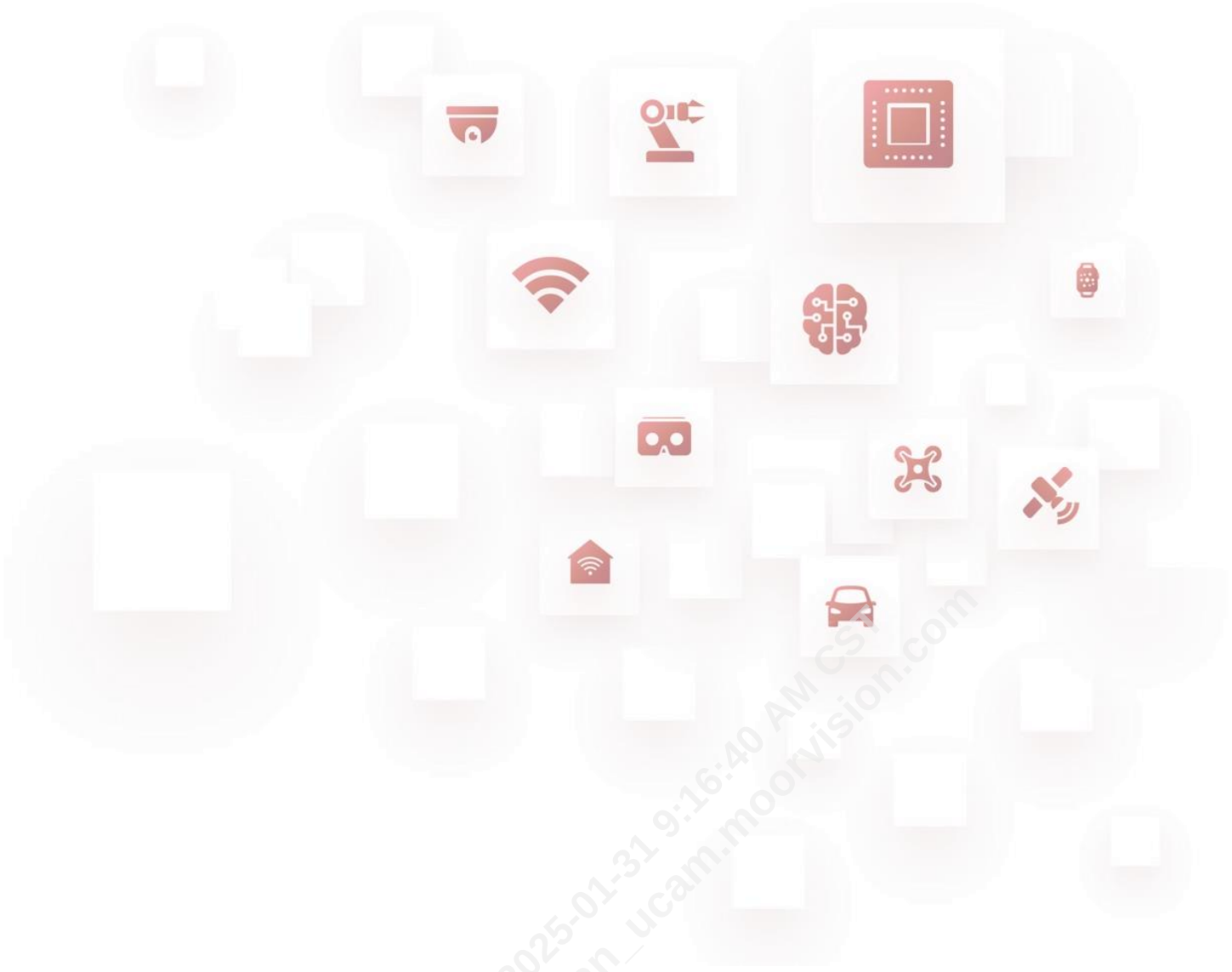
Thundercomm
AIOT ENABLER



2025-01-31 9:16:40 AM CST
Stefan_ucam.moonvision.com

Thundercomm TurboX C410/C610 Open Kit

# SNPE User Manual

Rev. V1.0
Dec 23, 2022

DN: tc-gle-23210

# Revision History

| Revision | Date | Description |
|---|---|---|
| 1.0 | Dec 23, 2022 | Initial release. |

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

Copyright© 2022 Thundercomm Technology Co., Ltd. All rights reserved.

# Table List

*Table 7-1. Supported Targets*

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

Copyright© 2022 Thundercomm Technology Co., Ltd. All rights reserved.

# About this document

- Illustrations in this documentation might look different from your product.
- Depending on the model, some optional accessories, features, and software programs might not be available on your device.
- Depending on the version of operating systems and programs, some user interface instructions might not be applicable to your device.
- Documentation content is subject to change without notice. Thundercomm makes constant improvements on the documentation of the products, including this guidebook.
- Function declarations, function names, type declarations, attributes, and code samples appear in a different format, for example, `cp armcc armcpp`.
- Code variables appear in angle brackets, for example, `<number>`.
- Button, tool, and key names appear in bold font, for example, click **Save** or press **Enter**.
- Commands to be entered appear in a different font; on the host computer use $ as shell prompt, while on the target device use # as shell prompt, for example,

```
$ adb devices
# logcat
```

- Part of the code that does not contain instructions appear in a different format, for example,

```
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0777", GROUP="adm"
```

- Folders, path and files are formatted in italic, for example, *turbox_flash_flat.sh*.

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

Copyright© 2022 Thundercomm Technology Co., Ltd. All rights reserved.

# Table of Contents

**Chapter 1. Overview ..... - 2 -**
1.1. General functions ..... - 2 -
1.2. Model workflow ..... - 3 -
**Chapter 2. Setup ..... - 4 -**
2.1. SNPE setup ..... - 4 -
2.2. Caffe and Caffe2 setup ..... - 7 -
2.3. Tensorflow setup ..... - 8 -
2.4. TFLite setup ..... - 8 -
2.5. TVM setup ..... - 8 -
2.6. PyTorch setup ..... - 9 -
2.7. ONNX setup ..... - 9 -
**Chapter 3. Network Models ..... - 10 -**
3.1. Model conversion ..... - 10 -
3.2. Model Quantization ..... - 10 -
**Chapter 4. Device Validation ..... - 11 -**
4.1. snpe-platform-validator ..... - 11 -
4.2. snpe-platform-validator-py ..... - 11 -
4.3. Validation output ..... - 12 -
**Chapter 5. Running Model ..... - 13 -**
5.1. On the host ..... - 13 -
5.2. On the device ..... - 14 -
5.3. Result ..... - 15 -
**Chapter 6. Benchmark ..... - 16 -**
6.1. Prerequisites ..... - 16 -
6.2. Run the benchmark ..... - 17 -
6.3. Generate signature file ..... - 18 -
**Chapter 7. Code Example ..... - 19 -**
7.1. Prerequisites ..... - 19 -
7.2. Build the C++ application ..... - 19 -
**Appendix 1. Notices ..... - 21 -**
**Appendix 2. Trademarks ..... - 23 -**

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
stefan.ucam.moovision.com

# Chapter 1. Overview

## 1.1. General functions

SNPE (Snapdragon Neural Processing Engine) is a Qualcomm® Snapdragon™ software accelerated runtime for execution of deep neural networks.

SNPE can enable users to perform the following tasks:

- Execute an arbitrarily deep neural network;
- Execute the network on the Snapdragon™ CPU, the Adreno™ GPU, the Hexagon™ DSP or AIP;
- Debug the network execution on x86 Ubuntu Linux;
- Convert Caffe, Caffe2, ONNXTM, TensorFlowTM, TFlite and Pytorch models to an SNPE DLC (Deep Learning Container) file;
- Quantize DLC files to 8-bit fixed point for running on the Hexagon™ DSP;
- Debug and analyze the performance of the network with SNPE tools;
- Integrate a network into applications and other codes via C++ or Java.

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

## 1.2. Model workflow

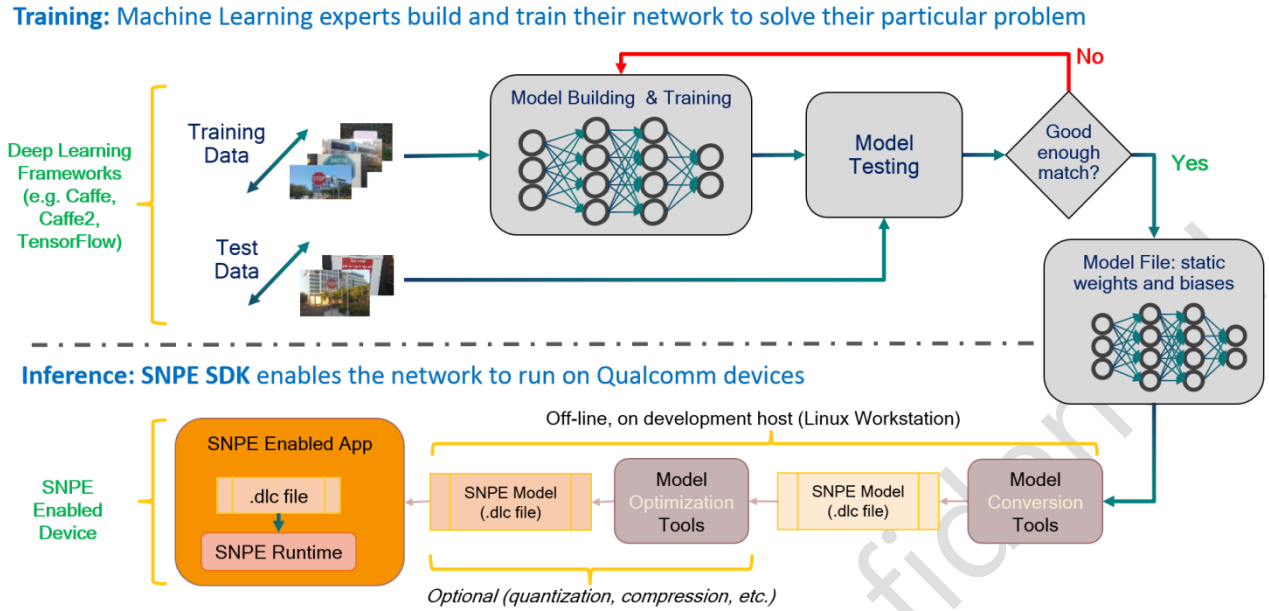



*Figure 1-1.Model Workflow*

Model training is performed on a popular deep learning framework (Caffe, Caffe2, ONNX, TensorFlow, TFlite and Pytorch models are supported by SNPE.) After training is completed, the trained model is converted into a DLC file that can be loaded into the SNPE runtime. This DLC file can then be used to perform forward inference passes using one of the Snapdragon accelerated compute cores.

The basic SNPE workflow only consists of several steps:

Step 1.Convert the network model to a DLC file that can be loaded by SNPE.

Step 2.Optionally quantize the DLC file for running on the Hexagon DSP and AIP.

Step 3.Prepare input data for the model.

Step 4.Load and execute the model using SNPE runtime.

Thundercomm Confidential
2025-01-31 9:16:10 AM CST
Stefan_ucam.msm@vision.com

# Chapter 2. Setup

## 2.1. SNPE setup

### 2.1.1. Prerequisites

- Currently the SNPE SDK development environment is limited to Ubuntu, specifically version 18.04.
- The SDK requires either Caffe, Caffe2, ONNX, PyTorch, TensorFlow or TFLite.
  - Instructions for Caffe: Caffe and Caffe2 Setup.
  - Instructions for Tensorflow: Tensorflow Setup.
  - Instructions for ONNX: ONNX Setup.
  - Instructions for TFLite: TFLite Setup.
  - Instructions for PyTorch: PyTorch Setup.
- Python 3.6

  Step 1.Download the latest version of Anaconda.

  Step 2.Execute the following commands.

  ```
  $ chmod +x Anaconda3-<version>-Linux-x86_64.sh
  $ bash Anaconda3-<version>-Linux-x86_64.sh
  ```

  Step 3.Enter the installation path of Anaconda and add the environment variables for anaconda in *~/.bashrc.*. If you do not change it, the default path is "*~/anaconda3*".

  ```
  $ export PATH=~/anaconda3/bin:$PATH
  $ source ~/.bashrc
  ```

  Step 4.Verify whether the installation is successful.

  ```
  $ conda -h 2>/dev/null && echo "Success" || echo "Failure"
  ```

  Step 5.Create a Python virtual environment.

  ```
  $ conda create -n snpe python=3.6
  ```

  Step 6.Enter the virtual environment.

  ```
  $ source activate snpe
  ```

  Afterwards, all SNPE work will be done in the virtual environment.

  ➲ **NOTE**: If you need to exit the virtual environment, execute the following command:

  ```
  $ source deactivate
  ```

- Android NDK (optional)

  Android NDK (android-ndk-r21c-linux-x86) is optional. The android-ndk-r23b version cannot be used. It is only required to build the native CPP example that ships with the SDK.

- Android SDK (optional)

  Android SDK (SDK version 23 and build tools version 23.0.2) is optional. It is only required to build the Android APK that ships with the SDK.

Thundercomm Confidential
2025-01-31 19:16:40 Arvest
Stefan_ucam.hoonvision.com

### 2.1.2. Download SDK

Download and unzip Qualcomm's SDK (1.54 and later.)

### 2.1.3. Check SDK environment and dependencies

Follow the steps below to set up the SDK environment.

Step 1. Extract the SDK to a directory that is referred to as *$SNPE_ROOT*.

```
$ unzip -X snpe-X.Y.Z.zip
```

➲ **NOTE**: "X.Y.Z" in this section refers to the release zip file that is in use (for example, snpe-1.54.0.zip).

Step 2. Run the dependencies script to check the system for Ubuntu package dependencies and install the missing packages.

```
$ source snpe-X.Y.Z/bin/dependencies.sh
```

Step 3. Check Python dependency.

Run the Python dependency checker to check the system for Python package dependencies and install the missing packages.

```
$ source snpe-X.Y.Z/bin/check_python_depends.sh
```

➲ **NOTE:** Refer to the following Python package versions tested with the SNPE SDK:

- numpy v1.16.5
- sphinx v2.2.1
- scipy v1.3.1
- matplotlib v3.0.3
- skimage v0.15.0
- protobuf v3.6.0(caffe2 requires v3.19.1)
- pyyaml v5.1

### 2.1.4. Set up environment

To build the Native C++ Example on Android, you need to execute the following command to export the Android NDK installed with the ANDROID_NDK_ROOT:

```
$ export ANDROID_NDK_ROOT=<path_to_ndk>
```

➲ **NOTE:** If the ANDROID_NDK_ROOT is not set, the environment setup script will attempt to find the NDK path by searching for ndk-build. Android NDK requires the version of android-ndk-r21b; the version of android-ndk-r23b cannot be used.

You can configure your model development environment as required.

- Set up the environment for Caffe.

  1) Using the SNPE SDK with Caffe models requires a valid Caffe installation (see 2.2.1 Caffe Setup).

  2) Go to *$SNPE_ROOT* and run the following command to set up the SNPE SDK environment.

```
$ source bin/envsetup.sh -c $CAFFE_DIR
```

Thundercomm Confidential
Steven 2025-01-31 9:16:40 AM CST ucam.moorvision.com

➲ **NOTE:**

- The location of the Caffe directory must be provided.
- *$CAFFE_DIR* is the installation path for Caffe.
- The script may show warnings about mismatched Caffe SHA.
- The SHA that the script checks for is the SHA that the SDK has been tested with.
- It is fine to use a different version of Caffe, including a version built from source.

- Set up the environment for Caffe2.

  1) Using the SNPE SDK with Caffe2 models requires a valid Caffe2 installation (see 2.2.2. Caffe2).

  2) Go to *$SNPE_ROOT* and run the following command to set up the SNPE SDK environment.

```
$ source bin/envsetup.sh -f $CAFFE2_DIR
```

  ➲ **NOTE:**

  - The location of the Caffe2 directory must be provided.
  - $CAFFE2_DIR is the installation path for Caffe2.

- Set up the environment for TensorFlow.

  1) Using the SNPE SDK with TensorFlow models requires a valid TensorFlow installation (see 2.3 TensorFlow setup).

  2) Go to *$SNPE_ROOT* and run the following command to set up the SNPE SDK environment.

```
$ source bin/envsetup.sh -t $TENSORFLOW_DIR
```

  ➲ **NOTE:**

  - The location of the TensorFlow directory must be provided.
  - *$TENSORFLOW_DIR* is the installation path for TensorFlow.
  - You can view the location with `pip show tensorflow`.

- Set up the environment for ONNX.

  1) Using the SNPE SDK with ONNX models requires a valid ONNX installation (see 2.7 ONNX setup).

  2) Go to *$SNPE_ROOT* and run the following script to setup the SNPE SDK environment.

```
$ source bin/envsetup.sh -o $ONNX_DIR
```

  ➲ **NOTE:**

  - The location of the ONNX directory must be provided.
  - *$ONNX_DIR* is the installation path for ONNX.

- Set up the environment for TFLite.

  1) Using the SNPE SDK with TFLite models requires a valid TFLite installation (see 2.4 TFlite setup).

  2) Go to *$SNPE_ROOT* and run the following command to set up the SNPE SDK environment.

```
$ source bin/envsetup.sh --tflite $TFLITE_DIR
```

- Set up the environment for PyTorch.

  1) Using the SNPE SDK with PyTorch models requires a valid PyTorch installation (see 2.6 PyTorch setup).

  2) Go to *$SNPE_ROOT* and run the following command to set up the SNPE SDK environment.

```
$ source bin/envsetup.sh -p $PYTORCH_DIR
```

Thundercomm Confidential

## 2.2. Caffe and Caffe2 setup

### 2.2.1. Caffe

A Caffe installation is required if the SNPE SDK is to be used with Caffe.

Clone and build Caffe from open source (https://github.com/BVLC/caffe) with the following commands.

```
$ sudo apt-get install cmake git libatlas-base-dev libboost-all-dev
libgflags-dev libgoogle-glog-dev libhdf5-serial-dev libleveldb-dev liblmdb-dev
libopencv-dev libprotobuf-dev libsnappy-dev protobuf-compiler python-dev python-numpy
```

➲ **NOTE**: Protobuf version must be 3.6.1 or below, otherwise, compiling may fail.

```
$ git clone https://github.com/BVLC/caffe.git
$ cd <path_to_caffe>
$ git reset --hard 18b09e807a6e146750d84e89a961ba8e678830b4
$ cp Makefile.config.example Makefile.config
$ make all
$ make test
$ make runtest
$ make distribute
$ make pycaffe
```

➲ **NOTE**: `$ make pycaffe` is crucial as the snpe-caffe-to-dlc conversion script relies on Caffe Python bindings.

For the installation process, refer to:

- http://caffe.berkeleyvision.org/install_apt.html
- http://caffe.berkeleyvision.org/installation.html#compilation

### 2.2.2. Caffe2

Caffe2 is a rapidly evolving project. You may install Caffe2 from prebuilt binaries for Ubuntu or build from sources.

Step 1.Execute the following commands to install from the source code.

```
$ sudo apt-get install -y --no-install-recommends build-essential git
libgoogle-glog-dev libgtest-dev libiomp-dev libleveldb-dev liblmdb-dev
libopencv-dev libopenmpi-dev libsnappy-dev libprotobuf-dev openmpi-bin
openmpi-doc protobuf-compiler python-dev python-pip

$ pip install --user future numpy protobuf typing hypothesis

$ sudo apt-get install -y --no-install-recommends libgflags-dev cmake

$ git clone https://github.com/pytorch/pytorch.git && cd pytorch
$ git submodule update --init --recursive
$ python setup.py install
```

Step 2.Verify whether the installation is successful.

➲ **NOTE**: Caffe2 requires Protobuf version 3.19.1.

```
$ pip install protobuf==3.19.1
$ cd ~ && python -c 'from caffe2.python import core' 2>/dev/null && echo "Success"
|| echo "Failure"
```

- If the installation fails, try to set up the environment of PyTorch first, and then verify caffe2.
- If installation fails, get a better error message by running Python in your home directory and then run from caffe2.python to import core inside Python.
- If this fails with a message about failing to find caffe2.python or failing to find libcaffe2, check the information on how to install Caffe2 in Python.

## 2.3. Tensorflow setup

The SNPE SDK has been tested with TensorFlow v1.6 and Tensorflow v2.3.

➲ **NOTE**: snpe-tensorflow-to-dlc has been updated to support conversion of Tensorflow 2.3 models. While some TF 1.x models may convert successfully by using Tensorflow 2.3 as the conversion framework, it is generally recommended that the TF version for training the model shall be consistent with that for conversion. Some older 1.x models may not convert at all by using TF 2.3 and a TF 1.x instance may be required for successful conversion.

Install pip and Tensorslow with the commands below:

```
$ sudo apt install python-pip
$ pip install tensorflow==1.6
```

## 2.4. TFLite setup

SNPE SDK has been tested with TFLite v2.3.0.

Step 1. Install TFLite as a standalone Python module using https://pypi.org/project/tflite/2.3.0/.

Step 2. Ensure TFLite is in your PYTHONPATH by running the following test command:

```
$ python -c "import tflite"
```

➲ **NOTE**: TFLite Converter depends on TVM. Instructions for TVM setup are at 2.5. TVM setup.

## 2.5. TVM setup

The SNPE SDK has been tested with TVM Commit ID 0f4c0654. TVM depends on Python 3.6.

Follow the steps below to set up TVM.

Step 1. Install TVM on Ubuntu Linux.

Option 1. Observe the instructions at https://tvm.apache.org/docs/install/from_source.html to install TVM. Follow those instructions to install TVM on Ubuntu Linux.

Option 2. Refer to the code below for a quick reference.

Get TVM source and checkout to 0f4c0654

```
$ git clone --recursive https://github.com/apache/incubator-tvm tvm
$ cd tvm
$ git checkout 0f4c0654
$ git submodule update --init
$ mkdir build
$ cp cmake/config.cmake build
```

Step 2. Update LLVM in the config file.

Since it takes long time to build from source, you can perform with either of the following options:

Option 1. Use new LLVM installation.

1) Download the pre-built version of LLVM and unzip it to your desired location.

2) Edit build/config.cmake USE_LLVM

```
set(USE_LLVM /path/to/your/llvm/bin/llvm-config)
```

Thundercomm Confidential

Option 2. Use the existing LLVM installation to let CMake search for a usable version of LLVM.

Edit *build/config.cmake*.

```
set(USE_LLVM ON)
```

➲ **NOTE**: LLVM 4.0 or higher is needed for building with LLVM. The version of LLVM from default apt may be lower than 4.0.

Step 3. Build TVM and related libraries.

```
$ cd build
$ cmake ..
$ make -j4
```

Step 4. Set up the environment.

```
$ export PYTHONPATH=<tvm source dir>/python:$PYTHONPATH
$ export LD_LIBRARY_PATH=<tvm build dir>:$LD_LIBRARY_PATH
```

Step 5. Check TVM dependencies at https://tvm.apache.org/docs/install/from_source.html#python-dependencies.

➲ **NOTE**:

- Necessary dependencies:

```
$ pip3 install --user numpy decorator attrs
```

- Environment test:

```
$ python -c "import tvm"
```

## 2.6. PyTorch setup

A PyTorch installation is required if the SNPE SDK is to be used with PyTorch models.
The SNPE SDK has been tested with PyTorch v1.8.1

Step 1. Observe the instructions at https://pytorch.org/get-started/locally to install PyTorch as a standalone Python module.

Step 2. Ensure PyTorch is in your PYTHONPATH with the following test command:

```
$ python -c "import torch"
```

➲ **NOTE:** PyTorch Converter depends on TVM. Instructions for TVM setup are at 2.5 TVM setup.

## 2.7. ONNX setup

To use the SNPE SDK with ONNX, an ONNX must be present on the system.

Observe the instructions at https://github.com/onnx/onnx#installation to install ONNX on Ubuntu Linux.

Thundercomm Confidential

# Chapter 3. Network Models

## 3.1. Model conversion

This section takes TensorFlow model as an example to show the model transformation process. For model transformation of other frameworks, refer to https://developer.qualcomm.com/sites/default/files/docs/snpe/usergroup3.html.

Step 1.Get Inception_v3.

```
$ cd $SNPE_ROOT/models/inception_v3
$ python scripts/setup_inceptionv3.py -a . -d
```

Then *inception_v3_2016_08_28_frozen.pb* is downloaded to the *$SNPE_ROOT/models/inception_v3/tensorflow*.

For more uses of *script setup_inceptionv3.py*, refer to https://developer.qualcomm.com/sites/default/files/docs/snpe/tutorial_setup.html#tutorial_setup_inception_v3.

Step 2.Model conversion.

Machine Learning frameworks have specific formats for storing neural network models. SNPE supports these various models by converting them to a framework DLC format. The DLC file is used by the SNPE runtime for execution of the neural network.

The **snpe-tensorflow-to-dlc** tool converts a frozen TensorFlow model or a graph meta file into an equivalent SNPE DLC file. The following command will convert an Inception v3 TensorFlow model into a SNPE DLC file.

```
$ snpe-tensorflow-to-dlc \
--input_network
$SNPE_ROOT/models/inception_v3/tensorflow/inception_v3_2016_08_28_frozen.pb \
      --input_dim input "1,299,299,3" --out_node
"InceptionV3/Predictions/Reshape_1" \
      --output_path inception_v3.dlc
```

## 3.2. Model Quantization

Step 1.Input data for quantization.

1) Convert the format of the input data from *.jpeg* to *.raw*.

```
$ cd $SNPE_ROOT/models/inception_v3/
$ mkdir -p data/cropped
$ python scripts/create_inceptionv3_raws.py -i data/ -d data/cropped/ -s 299
```

2) Create list file.

```
$ python scripts/create_file_list.py -i data/cropped/ -o data/cropped/raw_list.txt
-e *.raw
$ python scripts/create_file_list.py -i data/cropped/ -o data/target_raw_list.txt
-e *.raw -r
```

Step 2.Quantize the model.

For example, the following commands will convert an *Inception v3 DLC* file into a quantized *Inception v3 DLC* file.

```
$ cd $SNPE_ROOT/models/inception_v3/
$ snpe-dlc-quantize  --input_dlc dlc/inception_v3.dlc \
--input_list data/cropped/raw_list.txt \
--output_dlc dlc/inception_v3_quantized.dlc
```

Thundercomm Confidential
2025-01-31 9:15:40 AM GMT
stefan_ucam.moovision.com

# Chapter 4. Device Validation

**snpe-platform-validator** and **snpe-platform-validator-py** can check SNPE compatibility/capability of a device.

## 4.1. snpe-platform-validator

**snpe-platform-validator** checks the SNPE compatibility/capability of a device. This tool runs on the device, rather than on the host. It requires a few additional files to be pushed to the device besides its own executable file.

Step 1.Push SNPE tools and libs.

```
$ export SNPE_TARGET_ARCH=aarch64-ubuntu-gcc7.5
$ adb shell "mkdir -p /data/local/tmp/platformValidator/bin"
$ adb shell "mkdir -p /data/local/tmp/platformValidator/lib"
$ adb shell "mkdir -p /data/local/tmp/platformValidator/dsp"
$ adb push $SNPE_ROOT/lib/$SNPE_TARGET_ARCH/* /data/local/tmp/platformValidator/lib
$ adb push $SNPE_ROOT/lib/dsp/* /data/local/tmp/platformValidator/dsp
$ adb push $SNPE_ROOT/bin/$SNPE_TARGET_ARCH/snpe-platform-validator \
/data/local/tmp/platformValidator/bin
```

Step 2.Validate the device.

```
$ adb shell
# export LD_LIBRARY_PATH=/data/local/tmp/platformValidator/lib/
# export
ADSP_LIBRARY_PATH='/system/lib/rfsa/adsp;/system/vendor/lib/rfsa/adsp;/usr/lib/rfsa/
adsp;\
/data/local/tmp/platformValidator/dsp/'
# /data/local/tmp/platformValidator/bin/snpe-platform-validator --runtime all \
--targetPath /data/local/tmp/platformValidator/output
```

## 4.2. snpe-platform-validator-py

**snpe-platform-validator-py** checks the SNPE compatibility/capability of a device. The output is saved in a CSV file in the "Output" directory. Basic logs are also displayed on the console.

Step 1.Set up SNPE.

```
$ cd $SNPE_ROOT
$ export ANDROID_NDK_ROOT=<path_to_ndk>
$ source bin/envsetup.sh -t $TENSORFLOW_DIR
```

Step 2.Validate the device.

```
$ snpe-platform-validator-py --runtime all --directory . --buildVariant
aarch64-ubuntu-gcc7.5
```

Thundercomm Confidential
2025-01-13 9:16:40 AM CST
steven_ucas@moorvision.com

## 4.3. Validation output

Validating the device with **snpe-platform-validator** and **snpe-platform-validator-py** will generate the same result:

```
WARNING: SNPE AIP runtime not available on this SoC
PF_VALIDATOR: DEBUG: Calling PlatformValidator->setRuntime
PF_VALIDATOR: DEBUG: Calling PlatformValidator->RuntimeCheck
PF_VALIDATOR: DEBUG: Building and running a simple Vector addition gpu program.
Unit Test on the runtime GPU: Passed.
SNPE is supported for runtime GPU on the device.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->IsRuntimeAvailable
Runtime GPU Prerequisites: Present.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->setRuntime
PF_VALIDATOR: DEBUG: Calling PlatformValidator->RuntimeCheck
PF_VALIDATOR: DEBUG: Testing for the support of DSP runtime.
PF_VALIDATOR: DEBUG: CPU side validation passed.
PF_VALIDATOR: DEBUG: starting calculator test
PF_VALIDATOR: DEBUG: Loading DSP stub: libcalculator.so
PF_VALIDATOR: DEBUG: Successfully loaded DSP library - 'libcalculator.so'.  Setting up
pointers.
PF_VALIDATOR: DEBUG: Success in executing the sum function
Unit Test on the runtime DSP: Passed.
SNPE is supported for runtime DSP on the device.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->IsRuntimeAvailable
PF_VALIDATOR: DEBUG: CPU side validation passed.
PF_VALIDATOR: DEBUG: starting calculator test
PF_VALIDATOR: DEBUG: Loading DSP stub: libcalculator.so
PF_VALIDATOR: DEBUG: Successfully loaded DSP library - 'libcalculator.so'.  Setting up
pointers.
PF_VALIDATOR: DEBUG: Success in executing the sum function
Runtime DSP Prerequisites: Present.
Skipping checks for AIP runtime.
```

The above result indicates that GPU and DSP are present (OK). If the result is missing, it indicates that the device is NG.

Thundercomm 2025-01-31 9:16:40 AM CST Stefan_ucam.moorvision.com

# Chapter 5. Running Model

The example C++ application in this tutorial is called **snpe-net-run**. It is a command line that executes a neural network with SNPE SDK APIs.

- Required parameters for **snpe-net-run**:
  - A neural network model (DLC file)
  - An input list file with paths to the input data
- Optional parameters to **snpe-net-run**:
  - Choice of GPU, DSP or AIP runtimes (default: CPU)
  - Output directory (default: *./output*)
  - Show help description

## 5.1. On the host

Step 1.Run the following commands.

```
$ cd $SNPE_ROOT/models/inception_v3/
$ snpe-net-run --container dlc/inception_v3_quantized.dlc --input_list
data/cropped/raw_list.txt
```

The following output will be displayed.

```
-------------------------------------------------------------------------------
Model String: N/A
SNPE v1.54.2.2899
-------------------------------------------------------------------------------
Processing DNN input(s):
snpe/snpe-1.54/models/inception_v3/data/cropped/chairs.raw
Processing DNN input(s):
snpe/snpe-1.54/models/inception_v3/data/cropped/plastic_cup.raw
Processing DNN input(s):
snpe/snpe-1.54/models/inception_v3/data/cropped/notice_sign.raw
Processing DNN input(s):
snpe/snpe-1.54/models/inception_v3/data/cropped/trash_bin.raw
```

After successful operation, *output/* directory will be generated in the current path.

Step 2.View the classification result with the following commands.

```
$ cd $SNPE_ROOT/models/inception_v3
$ python scripts/show_inceptionv3_classifications.py -i data/target_raw_list.txt -o
output \
-l data/imagenet_slim_labels.txt
```

The following classification result will be displayed:

```
cropped/chairs.raw       0.382222 832    studio couch
cropped/plastic_cup.raw    0.990612 648   measuring cup
cropped/notice_sign.raw    0.167454 459   brass
cropped/trash_bin.raw     0.684572 413    ashcan
```

Thundercomm Confidential 2020-01-31 9:16:40 AM CST steam_ucam@qti.qualcomm.com

## 5.2. On the device

Step 1.Push SNPE tools and libs.

```
$ export SNPE_TARGET_ARCH=aarch64-ubuntu-gcc7.5
$ adb shell "mkdir -p /data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/bin"
$ adb shell "mkdir -p /data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/lib"
$ adb shell "mkdir -p /data/local/tmp/snpeexample/dsp/lib"
$ adb push $SNPE_ROOT/lib/$SNPE_TARGET_ARCH/* \
/data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/lib
$ adb push $SNPE_ROOT/lib/dsp/* /data/local/tmp/snpeexample/dsp/lib
$ adb push $SNPE_ROOT/bin/$SNPE_TARGET_ARCH/snpe-net-run \
/data/local/tmp/snpeexample/$SNPE_TARGET_ARCH/bin
```

Step 2.Push inception_v3.

```
$ cd $SNPE_ROOT/models/inception_v3/
$ adb shell "mkdir -p /data/local/tmp/inception_v3/cropped"
$ adb push data/cropped/*raw /data/local/tmp/inception_v3/cropped
$ adb push data/target_raw_list.txt /data/local/tmp/inception_v3
$ adb push dlc/inception_v3.dlc /data/local/tmp/inception_v3
```

Step 3.Run the following commands on the device.

```
$ adb shell
# export LD_LIBRARY_PATH=/data/local/tmp/snpeexample/aarch64-ubuntu-
gcc7.5/lib:$LD_LIBRARY_PATH
# export ADSP_LIBRARY_PATH="/data/local/tmp/snpeexample/aarch64-ubuntu-
gcc7.5/lib/../../dsp\
/lib;/system/lib/rfsa/adsp;/usr/lib/rfsa/adsp;/system/vendor/lib/rfsa/adsp;/dsp;/etc
/images/dsp;"
# cd /data/local/tmp/inception_v3
# rm -rf output
# /data/local/tmp/snpeexample/aarch64-ubuntu-gcc7.5/bin/snpe-net-run --container
inception_v3.dlc \
 --input_list target_raw_list.txt
```

➲ **NOTE**: **snpe-net-run** has parameters --use_gpu' and '--use_dsp' for GPU/DSP, such as:

```
# snpe-net-run --container inception_v3.dlc --input_list target_raw_list.txt
# snpe-net-run --container inception_v3.dlc --input_list target_raw_list.txt --
use_gpu
# snpe-net-run --container inception_v3.dlc --input_list target_raw_list.txt --
use_dsp
```

The following log will be displayed:

```
--------------------------------------------------------------------------------
Model String: N/A
SNPE v1.54.2.2899
--------------------------------------------------------------------------------
Processing DNN input(s):
cropped/chairs.raw
Processing DNN input(s):
cropped/plastic_cup.raw
Processing DNN input(s):
cropped/notice_sign.raw
Processing DNN input(s):
cropped/trash_bin.raw
```

## 5.3. Result

Step 1.Pull the generated result (the generated output directory) to the host.

Step 2.Execute the classification commands:

```
$ cd $SNPE_ROOT/models/inception_v3
$ mkdir output
$ adb pull /data/local/tmp/inception_v3/output output
$ python scripts/show_inceptionv3_classifications.py -i data/target_raw_list.txt -o
output \
-l data/imagenet_slim_labels.txt
```

The following classification result will be generated:

```
cropped/chairs.raw      0.382222 832   studio couch
cropped/plastic_cup.raw  0.990612 648   measuring cup
cropped/notice_sign.raw  0.167454 459   brass
cropped/trash_bin.raw   0.684572 413    ashcan
```

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

# Chapter 6. Benchmark

The benchmark shipped in the SNPE SDK consists of a set of Python scripts that run a network on a target Android/Linux Embedded device and collect performance metrics. It uses executables and libraries found in the SDK package to run a DLC file on target, as well as a set of inputs for the network and a file that points to that set of inputs.

The input of the benchmark scripts is a configuration file in JSON format. The SDK ships with a configuration file for running the InceptionV3 model that is created in the SNPE SDK. SDK users are encouraged to create their own configuration files and use the benchmark scripts to run on the target to collect timing and memory consumption measurements.

The configuration file allows the user to specify:

- Name of the benchmark (i.e., InceptionV3)
- Host path for storing results
- Device paths to use (where to push the necessary files for running the benchmark)
- Device to run the benchmark on (only one device is supported per run)
- Hostname/IP of remote machine to which devices are connected
- Number of times to repeat the run
- Model specific information (name, location of DLC, location of inputs)
- SNPE runtime configuration(s) to use (combination of CPU, GPU, GPU_FP16 and DSP)
- Measurements to take ("mem" and/or "timing")
- Profiling level of measurements ("off", "basic", "moderate" or "detailed")

## 6.1. Prerequisites

- SNPE setup (source bin/envsetup...)
- *model.dlc*
- Input data (list)

➲ **NOTE:** When the following error log occurs on DSP, it is due to no signature.

```
INFO - snpe_bench: Run 1
2022-12-07 13:45:02,851 - ERROR - snpebm.snpebm_device: ['SNPEFactory error 908: Error
code: -2. rpc_error_code=0x72, line_no=334', 'The selected runtime is not available on
this platform. Continue anyway to observe the failure at network creation time.']
2022-12-07 13:45:02,852 - ERROR - snpebm.snpebm_device: ['error_code=908;
error_message=DSP runtime communication error. Error code: -2. rpc_error_code=0x72,
line_no=334; error_component=DSP Runtime; line_no=417; thread_id=-159218576',
'error_code=908; error_message=DSP runtime communication error. Error code: -2.
rpc_error_code=0x72, line_no=334; error_component=DSP Runtime; line_no=417; thread_id=-
159218576']
2022-12-07 13:45:02,852 - WARNING - snpe_bench: Failed to perform benchmark for
DSP_timing.
2022-12-07 13:45:02,852 - INFO - snpe_bench: Running on DSP_ub_float_timing
```

For more information, refer to 6.3. Generate signature file.

Thundercomm Confidential
2025-01-31 9:16:40 AM GMT
steven_ucam.moorvision.com

## 6.2. Run the benchmark

Step 1.Create the configuration file.

```
$ cd $SNPE_ROOT/benchmarks
$ vi inception_v3.json
```

Step 2.Copy the following contents into the *inception_v3.json* file.

```
{
    "Name":"inceptionV3",
    "HostRootPath": "inception_v3",
    "HostResultsDir":"inception_v3/results",
    "DevicePath":"/data/local/tmp/snpeexample",
    "Devices":["ac49d838"],
    "HostName": "localhost",
    "Runs":5,

    "Model": {
       "Name": "Inception_v3",
       "Dlc": "../models/inception_v3/dlc/inception_v3.dlc",
       "InputList": "../models/inception_v3/data/target_raw_list.txt",
       "Data": [
       "../models/inception_v3/data/cropped"
       ]
    },

    "Runtimes":["GPU", "CPU", "DSP"],
    "Measurements": ["timing"],
       "CpuFallback":true,
       "ProfilingLevel":"detailed"
}
```

➲ **NOTE**: Some paths can be modified as required.

Step 3.Run the benchmark.

```
$ python snpe_bench.py -c inception_v3.json -a -t ubuntu64_gcc75
```

Step 4.The result generated in the *$SNPE_ROOT/benchmark/inception_v3/results/latest_results* can be checked in **Excel** or **LibreOffice calc**.

| | avg (us) | max (us) | min (us) | runtime | avg (us) | n |
|---|---|---|---|---|---|---|
| Load | 105 | 106 | 104 | CPU | 114 | |
| Deserialize | 15115 | 15564 | 14666 | CPU | 14864 | |
| Create | 191015 | 191687 | 190344 | CPU | 190974 | |
| Init | 207346 | 208448 | 206244 | CPU | 207060 | |
| De-Init | 39702 | 39801 | 39604 | CPU | 39154 | |
| Create Network(s) | 13830 | 13840 | 13820 | CPU | 13818 | |
| RPC Init Time | 147131 | 147766 | 146497 | CPU | 147284 | |
| Snpe Accelerator Init Time | 146571 | 147185 | 145958 | CPU | 146743 | |
| Accelerator Init Time | 138285 | 138898 | 137673 | CPU | 138335 | |
| Total Inference Time | 22393 | 22413 | 22373 | CPU\|DSP | 22419 | |
| Forward Propagate | 22327 | 22346 | 22308 | CPU\|DSP | 22349 | |
| RPC Execute | 21602 | 21622 | 21583 | CPU\|DSP | 21591 | |
| Snpe Accelerator | 21097 | 21118 | 21076 | CPU\|DSP | 21097 | |
| Accelerator | 20433 | 20455 | 20412 | CPU\|DSP | 20435 | |
| Misc Accelerator | 16 | 16 | 16 | CPU\|DSP | 15 | |
| layer_000 (Name:input:0 Type:data) | 290 | 294 | 287 | DSP | 290 | |

*Figure 6-1.Check Result in Excel or LibreOffice calc*

We can see the initialization time, reasoning time, etc. of the Inception_v3 model in the CSV file.

➲ **NOTE:** For more information on CSV files, refer to https://developer.qualcomm.com/sites/default/files/docs/snpe/benchmarking.html.

## 6.3. Generate signature file

Step 1.Execute the following command to obtain the serial number.

```
$ adb shell cat /sys/devices/soc0/serial_number
```

Step 2.Convert the format of the result from decimal to hexadecimal.

```
Example: 305419896
Hexadecimal: 0x12345678
```

Step 3.Execute the following command to obtain the signature file and download Hexagon_SDK.

```
{Hexagon_SDK}\tools\elfsigner\elfsigner.py -t {hexadecimal serial number} -o output
Example: C:\Qualcomm\Hexagon_SDK\3.4.3\tools\elfsigner\elfsigner.py -t 0x12345678 -o
output
```

Step 4.Find the rfsa folder.

```
$ adb shell ls /vendor/lib/rfsa/adsp
```

Step 5.Push it to the rfsa folder.

```
NOTE: Push it to the first fold you found in above command
$ adb root && adb disable-verity && adb reboot
$ adb root && adb remount && adb shell mount -o remount,rw /
Example:
$ adb push output/testsig-0x12345678.so /vendor/lib/rfsa/adsp/
```

➲ **NOTE:** If you need batch signature, please contact us at service@thundercomm.com.

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

# Chapter 7. Code Example

This tutorial demonstrates how to build a C++ sample application that can execute neural network models on the PC or target device.

## 7.1. Prerequisites

- Set up SNPE.
- Obtain Inception_v3

## 7.2. Build the C++ application

### 7.2.1 Build and Run on x86 Linux and Linux Embedded

Step 1.Start by entering the *snpe-sample base* directory.

```
$ cd $SNPE_ROOT/examples/NativeCpp/SampleCode
```

➲ **NOTE:**

- Makefiles vary with the Linux platform.
- The *$CXX* would need to be set according to the target platform. Here is a table of the supported targets, and their corresponding settings for *$CXX* and the Makefiles to use.

***Table 7-1. Supported Targets***

| Target | Makefile | Possible CXX value | Output Location |
|---|---|---|---|
| aarch64-oe-linux (gcc 8.2) | Makefile.aarch64-oe-linux-gcc8.2 | aarch64-oe-linux-gnu-g++ | aarch64-oe-linux-gcc8.2 |
| aarch64-oe-linux (gcc 8.2) | Makefile.aarch64-oe-linux-gcc8.2 | aarch64-oe-linux-gnu-g++ | aarch64-oe-linux-gcc8.2 |
| arm-oe-linux (gcc 8.2hf) | Makefile.arm-oe-linux-gcc8.2hf | arm-linux-gnueabihf-g++ | arm-linux-gnueabihf |
| arm-oe-linux (gcc 8.2hf) | Makefile.arm-oe-linux-gcc8.2hf | arm-linux-gnueabihf-g++ | arm-linux-gnueabihf |

Step 2.To compile sample for 610LE platform, you need to use Makefile.aarch64-ubuntu-gcc7.5 file. So we need aarch64-linux-gnu-g++ (7.5).

```
$ which aarch64-linux-gnu-g++
```

Step 3.Then you get the path of aarch64-linux-gnu-g++ (such as path/to/g++), and:

```
$ export CXX=/path/to/g++
$ make -f Makefile.aarch64-ubuntu-gcc7.5
```

Step 4.The generated snpe-sample tool is available in:
*$SNPE_ROOT/examples/NativeCpp/SampleCode/obj/local/aarch64-ubuntu-gcc7.5*.

The function of anpe-sample is similar to that of snpe-net-run. You can also modify the main.cpp file as desired.

Thundercomm Confidential

### 7.2.2 Build and run on ARM Android

You will need the Android NDK to build the Android C++ executable. It is assumed that you can invoke 'ndk-build' from the shell.

Step 1. Move to the base directory of snpe-sample.

```
$ cd $SNPE_ROOT/examples/NativeCpp/SampleCode
```

Step 2. Run the commands below to build snpe-sample with clang/libc++ SNPE binaries (i.e., arm-android-clang6.0 and aarch64-android-clang6.0).

```
$ cd $SNPE_ROOT/examples/NativeCpp/SampleCode
$ ndk-build NDK_TOOLCHAIN_VERSION=clang APP_STL=c++_shared
```

Step 3. The ndk-build commands will build both armeabi-v7a and arm64-v8a binaries of snpe-sample.

*$SNPE_ROOT/examples/NativeCpp/SampleCode/obj/local/armeabi-v7a/snpe-sample*

*$SNPE_ROOT/examples/NativeCpp/SampleCode/obj/local/arm64-v8a/snpe-sample*

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

# Appendix 1. Notices

Thundercomm may have patents or pending patent programs covering subject matter described in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries to service@thundercomm.com.

THUNDERCOMM PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you.

Changes are made periodically to the information herein; these changes will be incorporated in new editions of the publication. To provide better service, Thundercomm reserves the right to improve and/or modify the products and software programs described in the manuals, and the content of the manual, at any time without additional notice.

The software interface and function and hardware configuration described in the manuals included with your development board or system on module might not match exactly the actual configuration of that you have purchased. For the configuration of the product, refer to the related contract (if any) or product packing list, or consult the distributor for the product sales. Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

The products described in this document are not intended for use in implantation or other life support applications where malfunction may result in injury or death to persons. The information contained in this document does not affect or change Thundercomm product specifications or warranties. Nothing in this document shall operate as an express or implied license or indemnity under the intellectual property rights of Thundercomm or third parties. All information contained in this document was obtained in specific environments and is presented as an illustration. The result obtained in other operating environments may vary.

The information of this document should not be as any invitation for offer or any advice to the visitors. Please consult the professional comments from the sales consultant prior to do any actions of invest or purchase.

Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Any references in this publication to non-Thundercomm Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this Thundercomm product, and use of those Web sites is at your own risk.Thundercomm shall not be responsible for the content of the third party.

Any performance data contained herein was determined in a controlled environment. Therefore, the result obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

This document is copyrighted by Thundercomm and the property right of the date mentioned in this document, including but not limited trademarks, patents, copyrights, trade name etc. is are not covered by any open-source license. Thundercomm may update this document at any time without notice.

Anyone doesn't have the right to amend, reprint, republication, reproduce, transmit, distribute or any other way to use this document in business or public purpose without the prior written consent by Thundercomm.

E-mail messages sent to Thundercomm via the Internet are not guaranteed to be completely

Thundercomm Confidential

secure.Thundercomm shall not be liable for any loss incurred by the surfer when transmitting any information over the Internet or for any loss incurred by Thundercomm when sending any information over the Internet at your request.

Thundercomm has all rights under other relevant exemptions provided by laws and regulations, and Thundercomm's failure to claim or delay in claiming such rights shall not be deemed to be a waiver of such rights by Thundercomm.

Thundercomm reserves the right of final interpretation of this document.

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com

# Appendix 2. Trademarks

Thundercomm, Thundercomm TurboX, TURBOX, Thundersoft turbox are trademarks of Thundercomm Corporation or its associate companies in China and/or other countries. Intel, Intel SpeedStep, Optane, and Thunderbolt are trademarks of Intel Corporation or its subsidiaries in the U.S. and/or other countries. Microsoft, Windows, Direct3D, BitLocker, and Cortana are trademarks of the Microsoft group of companies. Mini DisplayPort (mDP), DisplayPort, and VESA are trademarks of the Video Electronics Standards Association. The terms HDMI and HDMI High-Definition Multimedia Interface are trademarks or registered trademarks of HDMI Licensing LLC in the United States and other countries. Wi-Fi, Wi-Fi Alliance, WiGig, and Miracast are registered trademarks of Wi-Fi Alliance. USB-C is a registered trademark of USB Implementers Forum. All other trademarks are the property of their respective owners.

Thundercomm Confidential
2025-01-31 9:16:40 AM CST
Stefan_ucam.moorvision.com